

Benchmarking de Técnicas de Coloração em Grafos Planares

Equipe 04

Ismael S. da Silva, Jose G. A. de Paula, Matheus F. Trajano, Pedro H. M. da Silva,

Universidade Federal do Ceará

24 de junho de 2025



- 1 Fundamentação Teórica
- 2 Representações de Grafos
- 3 Força Bruta x Heurística de Grundy
- 4 Resultados de Desempenho
- 5 Conclusão
- 6 Referências Bibliográficas

1 Fundamentação Teórica

2 Representações de Grafos

3 Força Bruta x Heurística de Grundy

4 Resultados de Desempenho

5 Conclusão

6 Referências Bibliográficas

Fundamentação Teórica

- **Grafos planares** são grafos que podem ser desenhados no plano de forma que suas arestas não se cruzem, exceto nos vértices em que se encontram.
- **Coloração de grafos** é um processo de atribuição de cores aos vértices de um grafo de forma que dois vértices adjacentes (conectados por uma aresta) nunca recebam a mesma cor. O objetivo é minimizar o número total de cores utilizadas.

Por que Grafos Planares?

- Possuem propriedades teóricas bem definidas, como o Teorema das Quatro Cores, o que fornece um limite superior teórico.
- São estruturas *esparsas*, o que os torna ideais para testar a eficiência dos algoritmos em contextos práticos.

Como os Grafos Planares foram gerados?

- **Construção Inicial**

- Define-se $k = \lfloor \sqrt{n} \rfloor$
- Cria-se uma malha retangular com $k \times k$ vértices
- Resultado: grafo **planar**, com até k^2 vértices

- **Quando Remove Vértices**

(se $k^2 > n$)

- Remove-se os $k^2 - n$ vértices excedentes
- Também são removidas as arestas incidentes a eles
- A planaridade é preservada, pois não há cruzamento novo

- **Quando Adiciona Vértices**

(se $k^2 < n$)

- Adiciona-se $n - k^2$ novos vértices ao grafo
- Cada novo vértice é conectado a um vértice existente com grau < 4
- Esse limite evita aglomeração e mantém a planaridade

Importância da geração sintética

- Permite o controle do número de vértices e reprodutibilidade dos testes.
- Garante que o grafo está de acordo com os *limites teóricos esperados*, como não ultrapassar o grau 4 para manter a planaridade.
- Evita depender de grafos reais com ruído ou propriedades indesejadas.

Cenário real: Alocação de frequências em torres de celular

Imagine uma cidade com várias torres de celular instaladas em diferentes bairros. Cada torre transmite sinal em uma certa frequência. Se duas torres **próximas** usarem a mesma frequência, os sinais podem **interferir** entre si, prejudicando a conexão.



Como modelar isso com grafos?

- Cada torre de celular é representada por um **vértice**.
- Se duas torres estão próximas (na mesma região de cobertura), conectamos os vértices com uma **aresta**, porque elas **não podem usar a mesma frequência**.
- Atribuímos uma “**cor**” para cada vértice, representando uma **frequência de transmissão**. O objetivo é **minimizar** o número de frequências diferentes, mas garantindo que torres próximas **não tenham a mesma cor**.

- 1 Fundamentação Teórica
- 2 Representações de Grafos**
- 3 Força Bruta x Heurística de Grundy
- 4 Resultados de Desempenho
- 5 Conclusão
- 6 Referências Bibliográficas

Lista de Adjacência

Um grafo $G = (V, E)$ é representado por um arranjo de listas. Para cada vértice $u \in V$, há uma lista que contém todos os seus vizinhos.

$$\text{Adj}(u) = \{v \in V \mid (u, v) \in E\}$$

Lista de Adjacência

Um grafo $G = (V, E)$ é representado por um arranjo de listas. Para cada vértice $u \in V$, há uma lista que contém todos os seus vizinhos.

$$\text{Adj}(u) = \{v \in V \mid (u, v) \in E\}$$

Pontos Fortes

- Eficiência de Espaço: Ideal para grafos com poucas arestas (esparsos). O espaço usado é proporcional ao número de vértices e arestas: $O(|V| + |E|)$.

Lista de Adjacência

Um grafo $G = (V, E)$ é representado por um arranjo de listas. Para cada vértice $u \in V$, há uma lista que contém todos os seus vizinhos.

$$\text{Adj}(u) = \{v \in V \mid (u, v) \in E\}$$

Pontos Fortes

- Eficiência de Espaço: Ideal para grafos com poucas arestas (esparsos). O espaço usado é proporcional ao número de vértices e arestas: $O(|V| + |E|)$.
- Busca de Vizinhos Rápida: Listar todos os vizinhos de um vértice é muito eficiente.

Lista de Adjacência

Um grafo $G = (V, E)$ é representado por um arranjo de listas. Para cada vértice $u \in V$, há uma lista que contém todos os seus vizinhos.

$$\text{Adj}(u) = \{v \in V \mid (u, v) \in E\}$$

Pontos Fortes

- Eficiência de Espaço: Ideal para grafos com poucas arestas (esparsos). O espaço usado é proporcional ao número de vértices e arestas: $O(|V| + |E|)$.
- Busca de Vizinhos Rápida: Listar todos os vizinhos de um vértice é muito eficiente.

Lista de Adjacência

Um grafo $G = (V, E)$ é representado por um arranjo de listas. Para cada vértice $u \in V$, há uma lista que contém todos os seus vizinhos.

$$\text{Adj}(u) = \{v \in V \mid (u, v) \in E\}$$

Pontos Fortes

- Eficiência de Espaço: Ideal para grafos com poucas arestas (esparsos). O espaço usado é proporcional ao número de vértices e arestas: $O(|V| + |E|)$.
- Busca de Vizinhos Rápida: Listar todos os vizinhos de um vértice é muito eficiente.

Pontos Fracos

- Verificação de Aresta Lenta: Checar se a aresta (u, v) existe pode exigir a varredura de toda a lista de vizinhos de u .

Matriz de Adjacência

Um grafo com n vértices é representado por uma matriz quadrada M de dimensão $n \times n$.

$$M_{uv} = \begin{cases} 1 & \text{se existe a aresta } (u, v), \\ 0 & \text{caso contrário.} \end{cases}$$

Matriz de Adjacência

Um grafo com n vértices é representado por uma matriz quadrada M de dimensão $n \times n$.

$$M_{uv} = \begin{cases} 1 & \text{se existe a aresta } (u, v), \\ 0 & \text{caso contrário.} \end{cases}$$

Pontos Fortes

- Verificação de Aresta Instantânea: Saber se a aresta (u,v) existe é uma operação de tempo constante: $O(1)$.

Matriz de Adjacência

Um grafo com n vértices é representado por uma matriz quadrada M de dimensão $n \times n$.

$$M_{uv} = \begin{cases} 1 & \text{se existe a aresta } (u, v), \\ 0 & \text{caso contrário.} \end{cases}$$

Pontos Fortes

- Verificação de Aresta Instantânea: Saber se a aresta (u,v) existe é uma operação de tempo constante: $O(1)$.
- Adição/Remoção Rápida de Arestas: Alterar o valor na matriz de 0 para 1 (ou vice-versa) é muito rápido.

Matriz de Adjacência

Um grafo com n vértices é representado por uma matriz quadrada M de dimensão $n \times n$.

$$M_{uv} = \begin{cases} 1 & \text{se existe a aresta } (u, v), \\ 0 & \text{caso contrário.} \end{cases}$$

Pontos Fortes

- Verificação de Aresta Instantânea: Saber se a aresta (u,v) existe é uma operação de tempo constante: $O(1)$.
- Adição/Remoção Rápida de Arestas: Alterar o valor na matriz de 0 para 1 (ou vice-versa) é muito rápido.

Matriz de Adjacência

Um grafo com n vértices é representado por uma matriz quadrada M de dimensão $n \times n$.

$$M_{uv} = \begin{cases} 1 & \text{se existe a aresta } (u, v), \\ 0 & \text{caso contrário.} \end{cases}$$

Pontos Fortes

- Verificação de Aresta Instantânea: Saber se a aresta (u, v) existe é uma operação de tempo constante: $O(1)$.
- Adição/Remoção Rápida de Arestas: Alterar o valor na matriz de 0 para 1 (ou vice-versa) é muito rápido.

Pontos Fracos

- Alto Custo de Memória: O espaço é sempre $O(|V|^2)$, mesmo para grafos com poucas arestas, o que gera desperdício.

Exemplo: Sugestão de Amigos em Rede Social

O Problema: Para um usuário, encontrar seus "amigos de amigos" para sugerir novas conexões.

Exemplo: Sugestão de Amigos em Rede Social

O Problema: Para um usuário, encontrar seus "amigos de amigos" para sugerir novas conexões.

- **Modelo:** Um grafo com 1 milhão de vértices (usuários).
- **Característica:** Grafo *esparso*, onde cada usuário tem em média 200 amigos (arestas).

Exemplo: Sugestão de Amigos em Rede Social

O Problema: Para um usuário, encontrar seus "amigos de amigos" para sugerir novas conexões.

- **Modelo:** Um grafo com 1 milhão de vértices (usuários).
- **Característica:** Grafo *esparso*, onde cada usuário tem em média 200 amigos (arestas).

Abordagem 1: Usando Lista de Adjacências

- **Passo 1:** Acessar a lista do usuário inicial. Custo: ler 200 nomes.

Exemplo: Sugestão de Amigos em Rede Social

O Problema: Para um usuário, encontrar seus "amigos de amigos" para sugerir novas conexões.

- **Modelo:** Um grafo com 1 milhão de vértices (usuários).
- **Característica:** Grafo *esparso*, onde cada usuário tem em média 200 amigos (arestas).

Abordagem 1: Usando Lista de Adjacências

- **Passo 1:** Acessar a lista do usuário inicial. Custo: ler 200 nomes.
- **Passo 2:** Para cada um desses 200 amigos, acessar suas respectivas listas (com 200 nomes cada).

Exemplo: Sugestão de Amigos em Rede Social

O Problema: Para um usuário, encontrar seus "amigos de amigos" para sugerir novas conexões.

- **Modelo:** Um grafo com 1 milhão de vértices (usuários).
- **Característica:** Grafo *esparso*, onde cada usuário tem em média 200 amigos (arestas).

Abordagem 1: Usando Lista de Adjacências

- **Passo 1:** Acessar a lista do usuário inicial. Custo: ler 200 nomes.
- **Passo 2:** Para cada um desses 200 amigos, acessar suas respectivas listas (com 200 nomes cada).

Exemplo: Sugestão de Amigos em Rede Social

O Problema: Para um usuário, encontrar seus "amigos de amigos" para sugerir novas conexões.

- **Modelo:** Um grafo com 1 milhão de vértices (usuários).
- **Característica:** Grafo *esparso*, onde cada usuário tem em média 200 amigos (arestas).

Abordagem 1: Usando Lista de Adjacências

- **Passo 1:** Acessar a lista do usuário inicial. Custo: ler 200 nomes.
- **Passo 2:** Para cada um desses 200 amigos, acessar suas respectivas listas (com 200 nomes cada).

Custo Computacional

O número de operações é proporcional às conexões reais:

$$\approx 200 \times 200 = \mathbf{40.000 \text{ operações (rápido e eficiente)}}$$

Análise Comparativa e Conclusão

Abordagem 2: Usando Matriz de Adjacências

- **Custo de Memória:** Exigiria uma matriz de $10^6 \times 10^6$, ou seja, 10^{12} células. **Totalmente inviável!**

Análise Comparativa e Conclusão

Abordagem 2: Usando Matriz de Adjacências

- **Custo de Memória:** Exigiria uma matriz de $10^6 \times 10^6$, ou seja, 10^{12} células.
Totalmente inviável!
- **Custo de Computação:** Para achar os amigos de alguém, é preciso varrer 1 milhão de colunas.

$$\approx 200 \times 1.000.000 = \mathbf{200 \text{ milhões de operações}}$$

Análise Comparativa e Conclusão

Abordagem 2: Usando Matriz de Adjacências

- **Custo de Memória:** Exigiria uma matriz de $10^6 \times 10^6$, ou seja, 10^{12} células.
Totalmente inviável!
- **Custo de Computação:** Para achar os amigos de alguém, é preciso varrer 1 milhão de colunas.

$$\approx 200 \times 1.000.000 = \mathbf{200 \text{ milhões de operações}}$$

Análise Comparativa e Conclusão

Abordagem 2: Usando Matriz de Adjacências

- **Custo de Memória:** Exigiria uma matriz de $10^6 \times 10^6$, ou seja, 10^{12} células. **Totalmente inviável!**
- **Custo de Computação:** Para achar os amigos de alguém, é preciso varrer 1 milhão de colunas.

$$\approx 200 \times 1.000.000 = \mathbf{200 \text{ milhões de operações}}$$

Tabela Comparativa

Critério	Lista de Adjacências	Matriz de Adjacências
Memória	Mínima ($O(V + E)$)	Gigantesca ($O(V ^2)$)
Velocidade	Quase Instantâneo	Extremamente Lento

Análise Comparativa e Conclusão

Abordagem 2: Usando Matriz de Adjacências

- **Custo de Memória:** Exigiria uma matriz de $10^6 \times 10^6$, ou seja, 10^{12} células. **Totalmente inviável!**
- **Custo de Computação:** Para achar os amigos de alguém, é preciso varrer 1 milhão de colunas.

$$\approx 200 \times 1.000.000 = \mathbf{200 \text{ milhões de operações}}$$

Tabela Comparativa

Critério	Lista de Adjacências	Matriz de Adjacências
Memória	Mínima ($O(V + E)$)	Gigantesca ($O(V ^2)$)
Velocidade	Quase Instantâneo	Extremamente Lento

Conclusão: Para grafos esparsos do mundo real, a **Lista de Adjacências** é a escolha mais adequada.

Exemplo de IA: Análise de Relações em Textos

O Problema: Um modelo de IA (como os que equipam a busca do Google ou o ChatGPT) precisa aprender a relação semântica entre as palavras mais importantes de um idioma.

Exemplo de IA: Análise de Relações em Textos

O Problema: Um modelo de IA (como os que equipam a busca do Google ou o ChatGPT) precisa aprender a relação semântica entre as palavras mais importantes de um idioma.

- **Modelo:** Um grafo com os **5.000 termos** mais comuns de um vocabulário.
- **Característica:** Grafo *denso*. Palavras como "governo" ou "economia" se conectam com milhares de outras.
- **Operação Crítica:** Durante o treinamento, o modelo precisa verificar milhões de vezes e de forma **instantânea** se dois termos, como ('ciência', 'dados'), possuem uma conexão direta.

Exemplo de IA: Análise de Relações em Textos

O Problema: Um modelo de IA (como os que equipam a busca do Google ou o ChatGPT) precisa aprender a relação semântica entre as palavras mais importantes de um idioma.

- **Modelo:** Um grafo com os **5.000 termos** mais comuns de um vocabulário.
- **Característica:** Grafo *denso*. Palavras como "governo" ou "economia" se conectam com milhares de outras.
- **Operação Crítica:** Durante o treinamento, o modelo precisa verificar milhões de vezes e de forma **instantânea** se dois termos, como ('ciência', 'dados'), possuem uma conexão direta.

Abordagem 1: Usando Matriz de Adjacências

- **Estrutura:** Uma matriz 5000×5000 . O espaço de memória é alto, mas fixo e perfeitamente gerenciável para este porte.

Exemplo de IA: Análise de Relações em Textos

O Problema: Um modelo de IA (como os que equipam a busca do Google ou o ChatGPT) precisa aprender a relação semântica entre as palavras mais importantes de um idioma.

- **Modelo:** Um grafo com os **5.000 termos** mais comuns de um vocabulário.
- **Característica:** Grafo *denso*. Palavras como "governo" ou "economia" se conectam com milhares de outras.
- **Operação Crítica:** Durante o treinamento, o modelo precisa verificar milhões de vezes e de forma **instantânea** se dois termos, como ('ciência', 'dados'), possuem uma conexão direta.

Abordagem 1: Usando Matriz de Adjacências

- **Estrutura:** Uma matriz 5000×5000 . O espaço de memória é alto, mas fixo e perfeitamente gerenciável para este porte.
- **Verificação de Aresta:** Para checar a conexão ('ciência', 'dados'), o acesso é direto à célula `M[id_ciencia][id_dados]`.

Exemplo de IA: Análise de Relações em Textos

Custo Computacional

A verificação da existência de uma aresta é em tempo constante:

$$O(1)$$

Análise Comparativa e Conclusão (Grafos Densos)

Abordagem 2: Usando Lista de Adjacências (Neste Cenário)

- **Verificação de Aresta:** Para checar a conexão ('ciência', 'dados'), o algoritmo precisaria:
 - ① Ir para a lista de vizinhos da palavra 'ciência'.
 - ② Percorrer **toda a lista** (que pode ter milhares de palavras) para procurar por 'dados'.
- **Custo:** $O(\text{grau}(\text{ciência}))$. Em um grafo denso, isso significa **milhares de operações para uma única verificação**, o que tornaria o treinamento da IA extremamente lento.

Análise Comparativa e Conclusão (Grafos Densos)

Abordagem 2: Usando Lista de Adjacências (Neste Cenário)

- **Verificação de Aresta:** Para checar a conexão ('ciência', 'dados'), o algoritmo precisaria:
 - ① Ir para a lista de vizinhos da palavra 'ciência'.
 - ② Percorrer **toda a lista** (que pode ter milhares de palavras) para procurar por 'dados'.
- **Custo:** $O(\text{grau}(\text{ciência}))$. Em um grafo denso, isso significa **milhares de operações para uma única verificação**, o que tornaria o treinamento da IA extremamente lento.

Análise Comparativa e Conclusão (Grafos Densos)

Abordagem 2: Usando Lista de Adjacências (Neste Cenário)

- **Verificação de Aresta:** Para checar a conexão ('ciência', 'dados'), o algoritmo precisaria:
 - 1 Ir para a lista de vizinhos da palavra 'ciência'.
 - 2 Percorrer **toda a lista** (que pode ter milhares de palavras) para procurar por 'dados'.
- **Custo:** $O(\text{grau}(\text{ciência}))$. Em um grafo denso, isso significa **milhares de operações para uma única verificação**, o que tornaria o treinamento da IA extremamente lento.

Tabela Comparativa (Para este Cenário de IA)

Critério	Matriz de Adjacências	Lista de Adjacências
Verif. de Aresta	Instantâneo ($O(1)$)	Lento ($O(\text{grau})$)
Uso em IA	Ideal para treino	Ineficiente para treino

Análise Comparativa e Conclusão (Grafos Densos)

Abordagem 2: Usando Lista de Adjacências (Neste Cenário)

- **Verificação de Aresta:** Para checar a conexão ('ciência', 'dados'), o algoritmo precisaria:
 - 1 Ir para a lista de vizinhos da palavra 'ciência'.
 - 2 Percorrer **toda a lista** (que pode ter milhares de palavras) para procurar por 'dados'.
- **Custo:** $O(\text{grau}(\text{ciência}))$. Em um grafo denso, isso significa **milhares de operações para uma única verificação**, o que tornaria o treinamento da IA extremamente lento.

Tabela Comparativa (Para este Cenário de IA)

Critério	Matriz de Adjacências	Lista de Adjacências
Verif. de Aresta	Instantâneo ($O(1)$)	Lento ($O(\text{grau})$)
Uso em IA	Ideal para treino	Ineficiente para treino

Conclusão: Para grafos **densos** e checagens de arestas em $O(1)$, a **Matriz de Adjacências** é a mais eficiente.

- 1 Fundamentação Teórica
- 2 Representações de Grafos
- 3 Força Bruta x Heurística de Grundy**
- 4 Resultados de Desempenho
- 5 Conclusão
- 6 Referências Bibliográficas

Algoritmo de Força Bruta

Funcionamento

Tenta todas as colorações possíveis de um grafo:

- Varia o número de cores k de 1 até n (número de vértices).
- Para cada k , testa todas as combinações possíveis de k cores para os n vértices.
- Finaliza quando encontra uma coloração válida.

Complexidade: Exponencial, para um grafo com n vértices e k cores, existem k^n colorações possíveis.

Algoritmo de Força Bruta

Pseudocódigo Algorítimo de força bruta:

```
Algoritmo ColorirForcaBruta(G)
n <-  de vertices de G
para k de 1 ate n faca
    para cada coloracao com k cores faca
        se a coloracao for valida entao
            retorne k, coloracao
        fim se
    fim para
fim para
retorne n, coloracao trivial
```

Listing 1: Implementação em pseudocódigo do algoritmo de força bruta

Heurística de Grundy

Funcionamento

Estratégia gulosa que percorre os vértices e atribui a menor cor disponível.

- Visita os vértices em uma ordem definida.
- Atribui a menor cor não usada pelos vizinhos.
- Finaliza quando todos os vertices foram coloridos.

Observações:

- Não é ótimo.
- O *número de Grundy* depende da ordem dos vértices.

Complexidade:

- Linear: Aproximadamente $\mathcal{O}(n + m)$ para grafos com n vértices e m arestas.

Heurística de Grundy

Pseudocódigo do Algoritmo de Grundy:

```
Algoritmo ColorirGrundy(G)
n <- numero de vertices de G
cores <- vetor de tamanho n, inicializado com -1

para cada vertice v em G (em ordem fixa) faca
    vizCores <- conjunto de cores dos vizinhos de v
    cor <- menor inteiro nao presente em vizCores
    cores[v] <- cor
fim para

retorne maior valor em cores + 1, vetor cores
```

Listing 2: Implementação em pseudocódigo da heurística de Grundy

Comparativo: Força Bruta vs. Grundy

Tabela Comparativa:

Critério	Força Bruta	Grundy
Precisão	Garante mínimo	Não garante
Desempenho	Muito lento	Muito rápido
Complexidade	Exponencial	Linear ($\mathcal{O}(n + m)$)
Aplicabilidade Prática	Pouco viável	Altamente viável

A escolha do algoritmo é um fator essencial para equilibrar precisão e desempenho.

Estratégia de Execução Paralela com OpenMP

Para otimizar o tempo total da coleta de dados, utilizamos a biblioteca OpenMP do C++ para executar os benchmarks em paralelo.

- A carga de trabalho foi dividida em 3 tarefas independentes, uma para cada tamanho de grafo a ser testado.
- As três tarefas, correspondentes a:
 - **n = 500**
 - **n = 1000**
 - **n = 2000**
- foram executadas simultaneamente na mesma máquina, aproveitando os múltiplos núcleos do processador para reduzir o tempo total de espera.

Processador utilizado para Execução Paralela

Description: Intel UHD Graphics for 13th Gen Intel Processors

Class: Laptop

Socket: FCBGA1964

Total Cores: 10 Cores, 16 Threads

Performance Cores: 6 Cores, 12 Threads, 2.4 GHz Base, 4.6 GHz Turbo

Efficient Cores: 4 Cores, 4 Threads, 1.8 GHz Base, 3.4 GHz Turbo

Typical TDP: 55 W

TDP Down: 45 W

TDP Up: 157 W

Cache per CPU Package:

L1 Instruction Cache: 6 x 32 KB

L1 Data Cache: 6 x 48 KB

L2 Cache: 6 x 1280 KB

L3 Cache: 20 MB

Cache per Eff. CPU Package:

Eff. L1 Instruction Cache: 4 x 64 KB

Eff. L1 Data Cache: 4 x 32 KB

Eff. L2 Cache: 1 x 2048 KB

Memory Support: Max. Memory Size: 128 GB (Up to DDR5 4800 MT/s Up to DDR4 3200 MT/s)

Desempenho do Algoritmo de Força Bruta (nanossegundos)

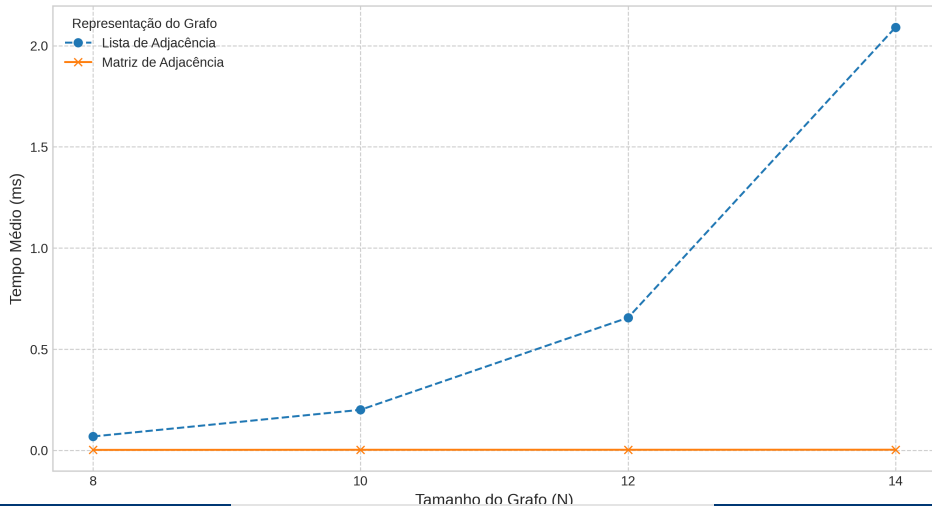
N (Vértices)	Lista (ns)	Matriz (ns)
8	68,833.60	2,428.82
10	200,551.00	3,104.70
12	655,799.00	3,146.28
14	2,090,960.00	3,384.53

Desempenho da Heurística de Grundy (milissegundos)

N (Vértices)	Lista (ms)	Matriz (ms)
500	5.34	47.92
1000	21.51	192.57
2000	90.64	566.79

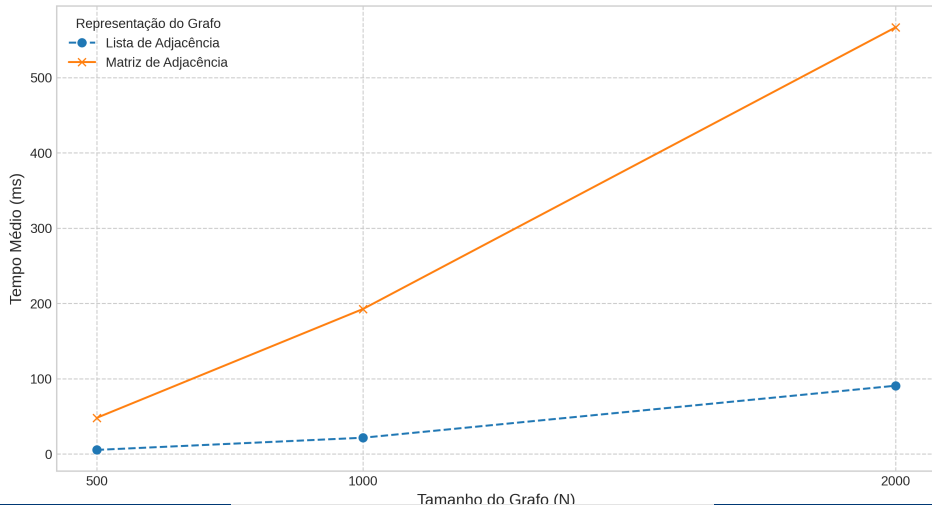
Desempenho do Algoritmo de Força Bruta por Representação

Desempenho do Algoritmo de Força Bruta por Representação



Desempenho do Algoritmo de Grundy por Representação

Desempenho do Algoritmo Heurístico de Grundy por Representação



Estimativas de Tempo para a Fase Grundy

- Para 1.000 repetições (10 vezes mais que 100):
 - $1,1 \text{ minutos} \times 10 = 11 \text{ minutos}$
- Para 10.000 repetições (100 vezes mais que 100):
 - $1,1 \text{ minutos} \times 100 = 110 \text{ minutos}$
 - Isso é aproximadamente 1 hora e 50 minutos.
- Para 1.000.000 de repetições (10.000 vezes mais que 100):
 - $1,1 \text{ minutos} \times 10.000 = 11.000 \text{ minutos}$
 - Convertendo para horas: $11.000/60 \approx 183,3 \text{ horas}$
 - Convertendo para dias: $183,3/24 \approx 7,6 \text{ dias}$

- 1 Fundamentação Teórica
- 2 Representações de Grafos
- 3 Força Bruta x Heurística de Grundy
- 4 Resultados de Desempenho
- 5 Conclusão**
- 6 Referências Bibliográficas

Conclusões & programação competitiva

- ❶ Inviabilidade da força bruta
- ❷ Heurísticas como solução adequada
- ❸ Lista de Adjacência como escolha preferencial
- ❹ Análise de restrições

- 1 Fundamentação Teórica
- 2 Representações de Grafos
- 3 Força Bruta x Heurística de Grundy
- 4 Resultados de Desempenho
- 5 Conclusão
- 6 Referências Bibliográficas**

Referências Bibliográficas I

- [1] CORMAN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: Teoria e Prática*. 3ª ed. Campus, 2012.
- [2] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4ª ed. Addison-Wesley Professional, 2011.
- [3] DIESTEL, R. *Graph Theory*. 5ª ed. Springer, 2017.
- [4] BONDY, J. A.; MURTY, U. S. R. *Graph Theory*. Springer, 2008.
- [5] SKIENA, S. S. *The Algorithm Design Manual*. 2ª ed. Springer, 2008.